How Happy Table Works

A tour of the restaurant app, explained without any computer words.

The guest's phone
Reads the menu
Puts food in a basket
Watches it being cooked

The waiter's tablet
Sees every table
Seats people
Adds up the bill

The kitchen screen
Shows the orders
Counts the minutes
Marks food ready

Imagine a restaurant. There are people eating, waiters walking about, and cooks in the kitchen. Each of them needs to know something different, and none of them wants to be shown the other two people's jobs.
That is the whole idea. The app is one building with three doors.

WHAT IS INSIDE

2 The journey of one plate of food
3 A map of the building
4 The one big switch
5 One app, many restaurants
6 Who is allowed to do what
7 How staff get in
8 Words grown-ups use

The journey of one plate of food

Follow a single dish from the table to the kitchen and back.

1. A guest sits down and scans the little code on the table.
2. The menu appears on their phone. They can look at all of it for free.
3. When they are ready, the waiter gives them a short table code.
4. The guest taps the food they want and sends the order.
5. The order pops up on the kitchen screen straight away.
6. The kitchen says how long it will take. The guest sees a countdown.
7. Food is marked ready. The ticket disappears from the screen.
8. At the end, the waiter closes the table and the bill is worked out.

Every one of those steps happens on a different screen, and all of them update at the same moment. When the kitchen marks food ready, the guest's phone changes by itself. Nobody presses refresh.

That trick has a name: **subscribing**. Instead of asking "is it ready yet?" over and over, a screen says "tell me when it changes", and then waits. Like putting your hand up once instead of shouting every ten seconds.

A map of the building

Inside the project there is a folder called **src**, short for source. Everything the app is made of lives in there, sorted into rooms.

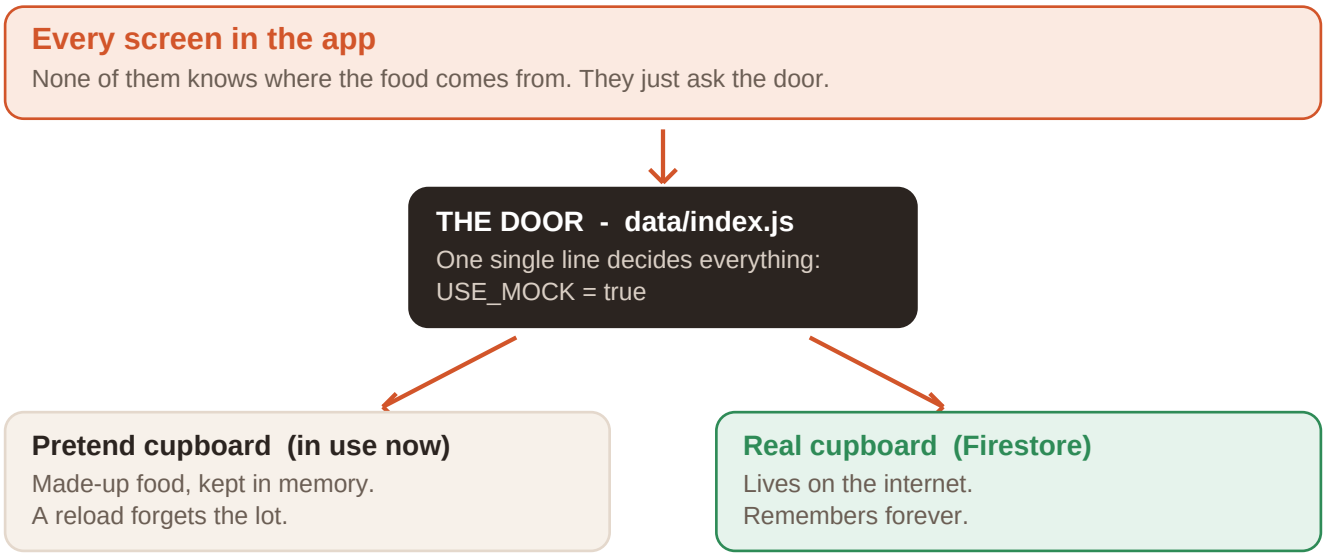
`routes/`	The rooms you walk into. One file per screen.
`components/`	The furniture. Buttons and cards, built once, used everywhere.
`hooks/`	The runners. They fetch the tables, the menu, the orders.
`data/`	The one door to the food cupboard. Everything goes through it.
`mock/`	A pretend cupboard, for practising.
`firebase.js`	The real cupboard, out on the internet.
`auth/`	The lock on the staff door, and who may open which cupboard.
`tenant/`	The sign over the door. Which restaurant is this?
`lib/`	The toolbox. Money, clocks, checking an email looks right.

The rule that keeps it tidy

A screen is never allowed to reach into the cupboard itself. It must ask through the door. That sounds fussy, but it is the reason the whole app can swap from pretend food to real food by changing one single word.

The one big switch

The cleverest thing in the project, and the easiest to explain.



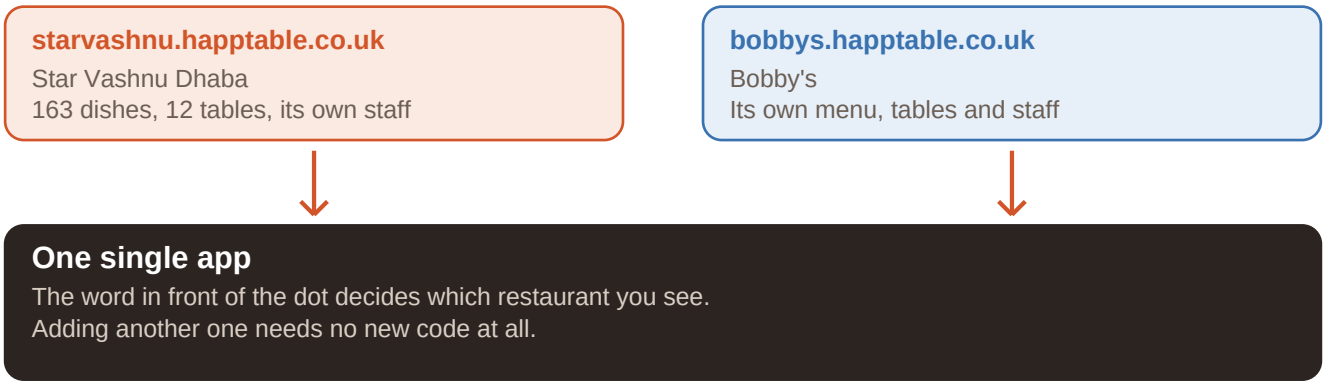
There are two cupboards. One is **pretend** — the food, the tables and the staff are all made up, and they live only in the computer's memory, so closing the page forgets them. The other is **real** — a proper database called Firestore, out on the internet, which remembers things forever.

Both cupboards have exactly the same handles. Ask either one for the menu and it hands you a menu. So the app genuinely cannot tell which one it is talking to, and that is the point: you can build and test the whole thing with pretend food, then flip one word and it is real.

Right now the switch is set to pretend. That is why you can break anything you like and a reload puts it all back.

One app, many restaurants

Two restaurants are already running on it, and neither can see the other.



This is called **multi-tenancy**, which only means "lots of tenants in one building". Each restaurant's food, tables, staff and orders are kept in a separate locked drawer. A waiter at one restaurant cannot sign in at the other, even with a perfectly good password, because they have no drawer there.

Adding a third restaurant needs no new code. It is one new row of information and one new word in front of the dot.

Who is allowed to do what

Four jobs. The owner decides what each one can touch.

Owner
Everything. Always.

Manager
Runs service. Not the money totals.

Waiter
The floor. Seating and tables.

Kitchen
The order board.

Those are only **starting points**, not rules carved in stone. The owner opens the People screen and sees a grid of switches: thirteen things a person might do, four kinds of person, and a switch at every crossing.

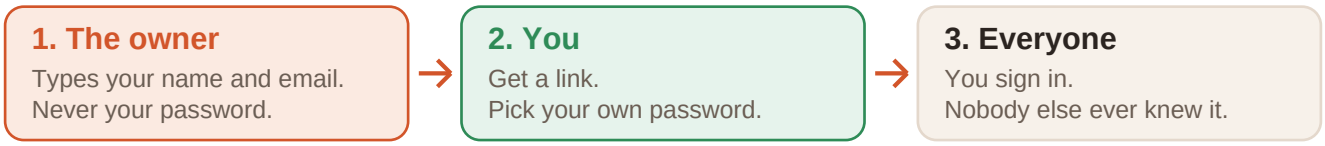
So "the manager can do everything except see how much money we took today" is a switch somebody flicks, not a thing a programmer has to rebuild. Flicking it changes every tablet in the building immediately — nobody has to sign out and back in.

The owner's own row is locked on purpose. There must always be one person who can undo a mistake, including a mistake made on that very screen.

One small kindness: a button you are not allowed to press is **hidden**, not greyed out. A screen full of things you can never do is just clutter.

How staff get in

The newest part of the app, and the one with a rule worth knowing.



The link only works once, and only for an hour. Ask for a new one and the old one stops working straight away.

The rule is this: **the owner creates the person, but never their password.**

It would be easier for the owner to type one in and read it out. That is exactly the problem. A password said out loud across a kitchen is a password four people now know, and it ends up written on a sticky note by the till. So instead the new person gets a link and chooses their own, and nobody else — not the owner, not the app — ever finds out what it is.

The same screen handles a forgotten password, because it is the same job. There is a small **Forgot your password?** link on the sign-in box, and it is deliberately the same link a brand new person uses on their first day.

Words grown-ups use

All of these mean something simpler than they sound.

Component
A piece of screen you build once and use everywhere, like a button. Build it well once instead of badly forty times.

Route
An address. /kitchen takes you to the kitchen screen the way a door number takes you to a house.

Hook
A little helper that fetches something a screen needs and keeps it up to date.

State
What is true right now. Which tables are full, what is in the basket, who is signed in.

Mock
A pretend version of something real, used for practising.

Firestore
The real cupboard. A database that lives on Google's computers and tells every screen the moment something changes.

Tenant
One restaurant inside the app that holds many restaurants.

Subscribe
Asking to be told when something changes, rather than checking again and again.

Build
Squashing all the files into a tiny bundle a browser can download quickly.

86
Kitchen word for "we have run out". The app uses it because every chef already does.

If you remember one thing

Every screen asks the same door for its food, and the door decides which cupboard to open. That is why this app can pretend and then be real without being rewritten.